EurekaZero: Learning to Invent by Self-Play

JopeDime Team

Draft — October 30, 2026

Abstract

Can experience from algorithm discovery improve the first proposal for a new problem? We frame invention as a game in which an encoded task is the board, an executable program is the move, a verifier supplies feedback and an incumbent defines improvement. EurekaZero aims to retain that feedback in a program writer, so deployment can use one complete candidate without retries or reranking. We separate this proposal budget from the costs of encoding, generation and fitting, and distinguish measured training behavior from the stronger claim of transfer to unseen problems. The available training record does not establish a single-proposal advantage. We specify comparisons against frontier-designed algorithms, a tabular foundation model and agents allowed repeated attempts, together with a test of the problem–program map. **[TBD: Complete the independent evaluations, identify the evaluated checkpoint and replace this draft statement with supported results and uncertainty.]**

1 Can Machines Invent?

Can a machine create knowledge that no human has written down? The strongest skeptics say no: a model trained on the record of human knowledge can only recombine it. Go suggests another answer. AlphaGo Zero, taught by a referee instead of a record, went beyond the best human play without seeing a human game [107]. We ask whether a similar training signal can improve algorithm invention: can learning from past evaluations make the first proposal on a new problem more useful?

The skeptics' challenge. In March 2025, Yann LeCun argued that language models retrieve existing solutions rather than find new ones: "in their purest form, they are incapable of inventing new things" [60]. The same month, Thomas Wolf warned that current trends will give us "a country of yes-men on servers" [125]. Dwarkesh Patel had asked Dario Amodei in August 2023 what to make of models with "basically the entire corpus of human knowledge memorized" that had not made "a single new connection that has led to a discovery" [84]. In January 2025, Demis Hassabis posed the hardest version as a benchmark for general intelligence: today's systems "can play a game of Go at a world champion level. But could a system invent Go?" [43]. These challenges separate producing an unfamiliar idea from learning which ideas will work. Predicting historical text does not directly reward improvement over an incumbent. That motivates a different training signal; it does not establish a ceiling on what a pretrained model can discover (Section 2).

Humans are bounded by history too. Research effort keeps rising while its productivity falls: Moore's law now needs more than 18 times as many researchers as in the early 1970s [8]. Under a growing burden of knowledge, inventors reach the frontier later and specialize more narrowly [48]. Papers and patents have grown less likely to break with the past [83], and fields open to outsiders when their leading scientists die [2]. The bottleneck is not a shortage of minds. It is the gravity of what we already know.

From search to learning. Automated research systems already couple proposals to tests. The AI Scientist [67] and the AI co-scientist [39] run agentic research loops; AlphaEvolve evolves programs against automated evaluators [77]; and Karpathy's autoresearch retains training changes that improve a measured objective [51]. Socher's *Eureka Machine* extends this vision to agents working with simulators and autonomous laboratories [110, 111]. These systems make judgment executable. Our question is what their evaluations can teach the system that proposes the next idea. Keeping a winning program preserves an output; training a proposer on those outcomes may preserve a capability that transfers to a new problem.

Key idea

Can judgment bought during training improve the first proposal at deployment?

The Zero move. Go faced the same choice and took the other road. AlphaGo learned from human expert games and searched heavily at every move [105]. AlphaGo Zero learned only from games against itself, and its network alone, with no search, rated 3,055 Elo. That is close to the 3,144 of AlphaGo Fan, the version that beat the European champion Fan Hui [105, 107]. This motivates an analogous goal for invention: learn from repeated program evaluations during training, then deploy a writer with a single candidate budget, search = 1 (Box 1). The budget is a test condition, not a promise that every first proposal wins. Figure 1 separates the training

Training	Propose → Evaluate → Update Repeated attempts; referee feedback can change the writer.
Deployment	Encode → Propose → Fit → Predict Frozen writer; one candidate committed before its score is known.

Figure 1: **Training feedback and the single-candidate contract.** The writer updates during training; the emitted estimator fits during deployment. Whether the learned writer improves the first proposal is the empirical question in Section 7.

feedback path from the frozen writer and fitting procedure at deployment.

Box 1: What "search = 1" means

- **One inference, one candidate.** For a new problem, the program writer emits one complete program ($B = 1$), with no retries, repairs, best-of-N, reranking or weight updates.
- **Everything is counted,** including reasoning tokens and problem encoding. One inference is one generation, not one forward pass.
- **Declare the fitting procedure.** Fitting the emitted algorithm to training rows is allowed. Any internal model or hyperparameter selection must be disclosed and counted in its compute budget. One outer proposal does not establish the absence of internal search. Training search is priced separately in Section 5.

A single candidate, with prior knowledge: the writer starts from pretraining and human ideas. The Zero analogy concerns the training signal; it does not imply that the system starts without human knowledge.

The proposal and its test. We formulate algorithm invention as a game: the encoded problem is the board, a complete program is the move, a verifier is the referee and the incumbent algorithm is the opponent (Section 4). EurekaZero combines a program writer with a problem–program map intended to guide learning from those contests (Section 6). The central hypothesis is that this training improves first proposals on unseen problems. The current learning record is reported separately from the proposed deployment comparisons.

Section 7 specifies the evidence needed to test that hypothesis: paired comparisons against algorithms proposed by Claude, Codex and Gemini; a comparison with KumoRFM [36]; a performance curve against agents allowed repeated attempts; and an ablation of the problem–program map. **[TBD: Complete these comparisons and document task coverage, model versions, prompts, budgets and uncertainty before claiming an advantage.]** The paper develops five elements:

1. **The anatomy of innovation:** four pillars (aim, variation, judgment and retention), with an idealized relation between proposal quality and the number of attempts (Section 2).
2. **The innovation game:** rules that make invention playable on many boards (Section 4).
3. **The economics of algorithmic self-play:** an accounting of training cost and the cost per verified improvement (Section 5).
4. **EurekaZero:** a writer and a problem–program map designed to learn from referee feedback (Section 6).
5. **An evaluation plan:** tests of first-proposal quality, deployment search and the map's contribution (Section 7).

Section 3 traces each pillar's history in AI; Section 8 asks what changes and where the claim stops.

To see what the referee can teach, we first separate the parts of innovation.

2 The Anatomy of Innovation

What must a system retain from an experiment to become a better inventor? We organize the process into four parts: an aim, a source of variation, a means of judgment and a way to retain what survives. Producing a new string is easy; finding a useful program is harder. We call judgment learned before the next attempt *taste*, and ask whether it can improve the first proposal.

2.1 Novelty is free

Novelty is the cheapest part of invention. Borges imagined a library holding every possible book of 410 pages [11], so every algorithm short enough to fit in a volume is already on its shelves. The library is useless, because nothing in it tells a reader which volume is true. A random string generator is the same library with a faster librarian: almost every string it emits is new, and almost none is worth running (Figure 2, lower right).

Language models show the same split. In a blinded study with 104 NLP researchers, ideas from a language model were rated more original than experts' ideas ($p < 0.05$) [103]. Once 43 experts had spent over 100 hours each carrying out ideas from both pools, the generated ideas lost significantly more ground on every metric [104]. Boden had predicted the asymmetry in 1998: machines, she wrote, would find generating new ideas easier than automating their evaluation [10]. These studies motivate a distinction between the novelty of a proposal and its value after execution; they do not establish that judgment is the only bottleneck in every domain.

2.2 The four pillars

Accounts of discovery from biology, psychology, philosophy and engineering provide a useful common vocabulary (Table 1). The four-pillar organization is our synthesis of these traditions, rather than a claim that their theories are identical.

Aim fixes the problem and what counts as better. Hamming asked colleagues, "What are the important problems of your field?" [42]. Pólya's first step is to understand the problem

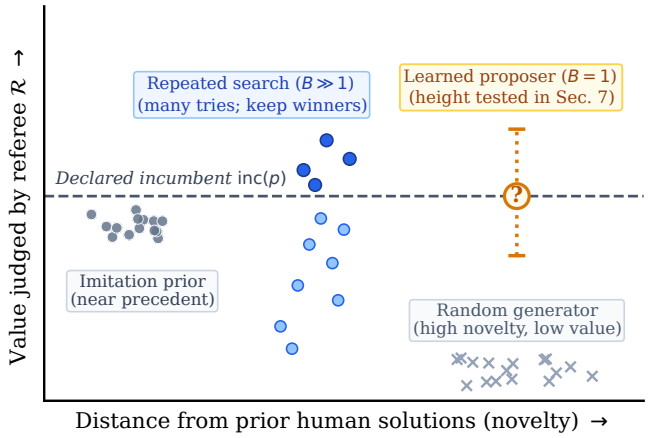


Figure 2: **Novelty is free; judgment is the inventor.** Schematic. Horizontal: distance from what people have already done. Vertical: value as judged by the referee. A random generator travels far and finds little. Imitation, repeated search and a learned proposer illustrate different ways to select candidates; their positions are conceptual, not measured limits. The value of EurekaZero's single proposal remains an empirical question for Section 7.

Table 1: **A four-part reading of traditions of discovery.** A dash marks a pillar outside a tradition's scope: Wallas analyzed a single act of thought. For Darwin and Campbell, the aim is a criterion set by the environment. Engineering is the design–build–test–learn cycle [76]; AlphaGo Zero follows Silver et al. [107]; EurekaZero is defined in Section 4.

Tradition	Aim	Variation	Judgment	Retention
Darwin [25]	Fitness to the environment	Variation	Natural selection	Inheritance
Wallas [123]	Preparation	Incubation, illumination	Verification	—
Campbell [15]	A selection criterion	Blind variation	Selection	Retention
Popper [91]	The problem	Conjecture	Refutation	The surviving theory
The scientific method	Question	Hypothesis	Experiment	Publication
Engineering	Design target	Design and build	Test	Learn
AlphaGo Zero	Win the game	Moves chosen with search	Game outcome	Network weights
EurekaZero	Problem encoding and metric	Program writer	Referee on hidden folds	Incumbent pool and weights

[89], and Popper argued that observation is always selective, so it presupposes a problem [91].

Variation proposes candidates. Peirce called abduction "the only logical operation which introduces any new idea" [85, CP 5.171]. Campbell modeled creative thought as blind variation [15, 109], and Koestler located creative acts in bisociation, the joining of two unrelated frames of thought [54]. Boden named three routes to a new idea: combining familiar ideas, exploring a conceptual space and transforming it [10].

Judgment tests a candidate against something outside the mind that proposed it: the attempted refutation [90], the experiment, peer review, and the referee of the common task framework, which scores every entry on data the competitors never see [30].

Retention preserves what survives: inheritance for Darwin [25], selective retention for Campbell [15], the textbook for a science and the incumbent for a game. It can store either an output or a way to propose better outputs. Archiving a winning program does the first; using evaluation feedback to update the generator attempts the second. This distinction connects the four pillars to the learning question in Section 1.

The convergence has a reason. Campbell argued that every genuine gain in knowledge needs a mechanism for introducing variation, a consistent selection process, and a mechanism for preserving and reproducing what is selected [15, p. 381]. The aim is what keeps the selection consistent, which makes it the fourth pillar.

2.3 Taste is judgment moved upstream

Skill in invention is judgment that arrives before the try. For Poincaré, creating meant not making the useless combinations: "Invention is discernment, choice." The sterile combinations, he added, do not even present themselves to the inventor [87, pp. 324–325]. Chess masters show the same economy: in de Groot's think-aloud studies they considered about as many moves as weaker players but found the right ones [26], because they see a position as a few large, familiar chunks [18]. Much of that knowledge is tacit; experts know more than they can say [88].

We call judgment learned in advance *taste*. AlphaGo's policy network is taste made explicit: a network that selects which moves are worth playing, trained first on human expert games and then refined by self-play [105]. Its first teacher was history.

Taste has a price we can state. Suppose each try beats the incumbent with a fixed, nonzero probability q, independently of the others. The number of tries up to and including the first win is then geometric, with mean

$$\mathbb{E}[N_{\text{tries}}] = \frac{1}{q}, \tag{1}$$

where N_{tries} counts the candidates the referee scores. Under this fixed-probability, independent-try model, better proposal quality shortens the expected search.[1]

The equation isolates the value of learning before deployment. It does not rank random generators, pretrained models and agent systems without measuring their proposal quality. In particular, search = 1 fixes the number of candidates a system may submit; it does not require the probability of a win to approach certainty. A single-candidate system can fail, and every failure must remain in its reported results. Section 5

[1] Adaptive search and correlated proposals need not follow Equation (1); independence alone does not say whether the approximation overstates or understates their cost. Empirical attempt curves are required for those systems.

uses the same idealization to relate evaluation cost to the cost of a verified improvement.

Key idea

Novelty is free. Judgment is the inventor.

2.4 Learning from a referee

Taste depends on the feedback available to the learner. People learn much of it from the record of their field. Normal science, in Kuhn's account, solves puzzles inside a reigning paradigm [57]. The record also holds people in place: when a star life scientist dies prematurely, articles by outsiders to the star's collaborators flow into the field, draw on different literature and are disproportionately likely to be highly cited [2].

A model trained to predict text receives a signal about what was written, rather than a direct score for improving an algorithm. That does not prevent novel generalization. It motivates supplementing the historical record with feedback from execution, so a useful departure from precedent can be rewarded.

A referee offers such an additional teacher. AlphaGo Zero learned with no human data, from the rules and the outcomes of its own games, and found strategies outside traditional Go knowledge [107]. Its predecessor had already made such a move. As David Silver told Wired in March 2016, AlphaGo put the chance that a human would play its Move 37 against Lee Sedol at one in ten thousand, and it won that game [71]. The knowledge then flowed back to people. In more than 5.8 million moves by professionals from 1950 to 2021, previously unseen moves grew more frequent once superhuman programs arrived, and came with better decisions [102]. These results establish the possibility in Go. Whether an analogous signal improves algorithm invention must be measured in the innovation game.

2.5 Where the pillars live

Machines do not remove the pillars; they move them (Table 2). The most ambitious swarm design fits the same anatomy. In an October 2026 essay drawn from his book, Socher rests the Eureka Machine on "a living map of all human knowledge", "a unified model of physical reality", "high-fidelity digital twins" and "fully autonomous physical laboratories" [110, 111]. On top of them, millions of AI agents critique each other's ideas. In our terms, the map is retention; the model of reality, the twins and the laboratories are faster and better judgment; and the agents supply variation and critique.

Agent search and learning from a referee are complementary. A search can retain programs, revise its context or update its proposer; calling it a swarm does not determine which of these it does. The comparison in Table 2 therefore isolates a common deployment setting in which search leaves the proposer's weights fixed. EurekaZero instead aims to use training evaluations to improve the proposer before a new problem arrives. Both approaches still need a reliable referee, and the cost of training must be counted alongside any saving at deployment.

Table 2: **Where feedback can be retained.** The agent-search column describes fixed-weight deployment loops, not every agent system. The EurekaZero column states the proposed training and deployment protocol. In every column, people still set the aim.

	Human science	Agent search	EurekaZero
Aim	The researcher chooses	A person writes the goal	Problem encoding and metric, chosen by people for now
Variation	The scientist's ideas	Agents propose and critique	One candidate per test problem
Judgment	Experiments, peer review	Experiments, simulators, autonomous labs	The referee, on hidden folds
Retention	The literature	Logs, archives, a map of knowledge	Incumbent pool and the writer's weights
Where taste lives	Years of training, in one head	Weights fixed; context can adapt	Training on referee feedback
Deployment budget	Varies by problem	Multiple adaptive attempts	One candidate; success measured

The next section traces these parts through AI research, asking how earlier systems connected evaluation to the ability to propose new solutions.

3 The Road to Zero: Five Building Blocks

What has AI already built for each of the four pillars of Section 2? Nearly all of it. Deep learning gave variation a voice, program evolution and reinforcement learning built judgment loops, games made judgment cheap, and the race for self-improvement now scales search to swarms of agents. These lines of work motivate a narrower question: can experience across algorithm-discovery tasks improve the first complete program proposed for a new family? The combination matters more than a claim that any one component is new.[2]

3.1 Deep learning gave variation a voice

Deep learning built the variation pillar: machines that propose. Learned representations replaced hand-written rules [61]. AlexNet made the case on images [56], the transformer made it general [120], and scaling laws made progress predictable [44, 50]. Sutton [114] drew the lesson: general methods that use computation win. Generative models then turned representations into proposals, from text to video [14] and code. Codex solved 28.8% of the HumanEval programming problems with one sample, and 70.2% with 100 samples, counting any that passed the unit tests [19]. That gap illustrates the value of a

[2] Each history keeps only the milestones the argument needs; Appendix A reserves space for an extended account.

larger sampling budget. It is not a direct estimate of the geometric model in Equation (1): pass rates and the probability of beating an incumbent measure different outcomes.

Networks already propose outputs assessed by task-specific tests: protein structures that rival experiment in most cases [49], 2.2 million crystals predicted to be stable [70], and halicin, an antibiotic that then treated infections in mice [112]. CLIP put images and captions in one shared space [92], the template for our problem–program map (Section 6).

What EurekaZero takes: representations that read a problem, models that write code, and one space for two kinds of object.
The question carried forward: how to train a code proposer directly on verified algorithm quality, and test transfer beyond its training tasks.

3.2 Machines that invent algorithms built the variation–judgment loop

Program evolution joined variation to judgment: propose a program, score it, keep the better ones. Friedberg [37] let a machine vary its own instructions, Holland [45] made variation and selection a general method of adaptation, and Lenat's AM and EURISKO searched concepts and heuristics by measures of interest [65, 66]. EURISKO takes its name from the Greek verb behind *eureka*, as ours does. Koza [55] closed the loop with genetic programming: a population of programs, a fitness function as referee and selection as coach. A population retains useful programs; a learned proposer can also retain patterns that help construct programs for later tasks.

Later work sharpened both halves: superoptimization [69], program synthesis from examples [40] guided by neural networks [5], neural architecture search [132], and AutoML-Zero, which evolved learning algorithms from basic mathematical operations [94]. Then language models became the variation operator. FunSearch found cap-set constructions beyond the best known [96]: an answer to the skeptics of Section 1, bought with search. Lion, an optimizer found by program search, went into production at Google [20]. AlphaEvolve multiplied two 4×4 complex matrices with 48 scalar multiplications, improving on Strassen's algorithm in that setting after 56 years [77]. MLEvolve carries the loop into machine-learning engineering [31].

What EurekaZero takes: programs as moves, and the generate–judge loop.
The question carried forward: how much of a verified search can transfer into the proposer. DreamCoder already learns reusable libraries and a search-guiding model across tasks [32]; a single-candidate evaluation tests how useful a learned proposal is before additional task-specific search.

3.3 Reinforcement learning turned retention into taste

Reinforcement learning turns a referee's verdicts into a better player. Bellman [6] gave it an equation of value. Samuel's checkers program improved by playing a copy of itself [98], TD-Gammon reached near the best human backgammon players by self-play [115], and DQN learned Atari games from pixels [72]. AlphaGo learned from human games and searched at every move [105]; AlphaGo Zero learned by self-play alone [107]. AlphaZero mastered three games with one algorithm [108], MuZero learned the rules as well [100], and AlphaStar's league of mutually exploiting agents reached Grandmaster level in StarCraft II [121].

The key concept is expert iteration [1]: search is the teacher, the network is the student, and in the end the student alone plays strongly. AlphaGo Zero's raw network, with no search, nearly matched the version that had beaten a professional (Section 1); Box 2 shows how. Language models now learn the same way, from human preferences [22, 82] and from rule-based checks of answers, as in DeepSeek-R1 [41].

What EurekaZero takes: the learning rule, and proof that a network can absorb search.
The question carried forward: how to define a reward for relative algorithm quality, then distinguish improvement on training games from improvement on unseen families.

Box 2 — Self-play in 90 seconds

A player starts with only the rules and plays itself. A short search before each move finds better moves than instinct would, and the referee scores each finished game. The network learns to propose the moves the search chose (its policy) and to predict the winner (its value), so the next search is stronger. Over millions of games, search is distilled into intuition [1, 107]. *What if the board were a problem, and the move an invention?*

3.4 Gamification made judgment cheap

Games built the judgment pillar at scale. Shannon [101] chose chess because its moves and its goal are sharply defined. The common task framework made machine-learning research a game: shared data, enrolled competitors and a referee who scores every entry on a sealed test set [30], as in the Netflix Prize [7] and ImageNet [97]. People played science too. Foldit players beat automated methods on hard protein puzzles [23] and solved a protease structure that earlier attempts had not [52]; EteRNA players, whose designs were tested in a laboratory, found RNA design rules [62].

Then machines played problems. AlphaTensor's TensorGame beat the best known matrix-multiplication algorithms for several sizes [35], and AlphaDev's AssemblyGame found sorting routines now in the LLVM C++ library [68]. AlphaGeometry learned olympiad geometry without human demonstrations [118]. AlphaProof, refereed by the Lean proof checker, reached silver-medal standard at the 2024 International Mathematical Olympiad alongside AlphaGeometry 2 [47].

What EurekaZero takes: the recipe for a game: rules, a referee, a score and a sealed test.
The question carried forward: which parts of a task can vary under a shared representation and referee. EurekaZero makes

the problem an input to the proposer (Section 4); whether that supports transfer across families remains a result to establish.

3.5 The race for self-improvement scaled search

The race for self-improvement scales all four pillars, so far with search. Good [38] foresaw an "intelligence explosion", and Schmidhuber [99] specified a Gödel machine that rewrites its own code once it proves the rewrite useful. STOP improves the scaffold that calls a language model, including itself, and its authors note: "Since the language models themselves are not altered, this is not full recursive self-improvement" [128]. The Darwin Gödel Machine's self-editing agents raised its SWE-bench score from 20.0% to 50.0% [129], and AlphaEvolve cut Gemini's training time by 1% [77].

Language models also play themselves. SPIN plays its previous iteration [21], Absolute Zero proposes and solves coding tasks under a code executor [130], and STP pairs a conjecturer with a Lean prover [29]. Such self-play plateaus as the conjecturer games its reward, Bailey et al. [3] report, unless a guide keeps it on track. Our closest neighbors learn to discover. EvoTune fine-tunes on what its own evolutionary search finds [113], and TTT-Discover trains on the one problem at hand, at a few hundred dollars per problem [127]. Evolution Fine-Tuning learns from 156,000 search trajectories across 371 tasks [63], and DiscoRL meta-learned a reinforcement-learning rule that beat human-designed ones [79]. These approaches retain improvements in different places: a search trajectory, a task-specific learner or a reusable learning rule. Our comparison asks about the quality and cost of the first complete candidate, rather than treating these forms of learning as absent.

Research itself is becoming a swarm. The AI Scientist automates ideas, experiments and papers [67], and the AI co-scientist ranks agents' hypotheses in an Elo tournament [39]. Karpathy's autoresearch, released in March 2026, runs about 100 experiments overnight and keeps what improves [51]. In May 2026, Recursive raised $650 million to build AI that improves itself [13]. Their referees are our boards (Section 2).

Self-improvement comes in three strengths: better external algorithms, better components, and a better ability to improve, with verified discoveries per unit of compute rising across generations. These require separate evidence (Section 8); a recorded training trajectory alone establishes none of them on an unseen task.

What EurekaZero takes: the destination, and the safety frame: verify every improvement before keeping it, and sandbox every program.
The question carried forward: whether learning across discovery episodes improves the first proposal, compared with the corresponding frozen proposer under the same deployment budget.

Table 3: **Related approaches retain experience in different forms.** This qualitative comparison separates output, deployment budget and retained state. Exact budgets depend on each study's protocol. The last row states EurekaZero's design; its proposal quality is still to be tested.

System	Output	Deployment budget	Retained experience	Feedback source
Genetic programming; AutoML-Zero	An algorithm	Study-specific search	Selected programs	Executable objective
AlphaTensor; AlphaDev	An algorithm for one family	Task-specific search	A policy for that game	Game reward
FunSearch; AlphaEvolve	An algorithm	Iterative search	Program archive	Automated evaluation
Research swarms (AI Scientist, autoresearch)	Experiments and code	Adaptive attempts	Context, logs and archives	Experimental outcomes
EvoTune; TTT-Discover; Evolution Fine-Tuning	An algorithm or construction	Protocol-specific search	Fine-tuned policy or search state	Search trajectories
TabPFN; MotherNet; KumoRFM	Predictions or network weights	Task-specific inference	Pretrained parameters	Training across tasks
Absolute Zero; STP	Answers and proofs	Decoding budget varies	Policy parameters	Correctness feedback
EurekaZero	**An algorithm**	**One candidate**	**Proposed writer updates**	**Incumbent comparisons**

3.6 Where the roads meet

Table 3 distinguishes what a system produces from how it retains experience. TabPFN, MotherNet and KumoRFM provide neighboring examples of learning across tasks [36, 46, 75]. Their prediction or network-generation interfaces differ from the proposed readable-program output of EurekaZero, but a common candidate budget alone would not make their computation or task assumptions equivalent. Absolute Zero and STP learn by self-play, with correctness as feedback. Our proposed comparison standard is improvement against a declared incumbent, which is why the game needs an opponent (Section 4).

Key idea

The test is whether experience on earlier discovery tasks improves the first complete proposal on a new one.

To build that player, we first make invention a game.

4 Innovation Is a Game

Can invention be played like Go? It can. The encoded problem is the board, an algorithm is the move, a verifier is the referee and a declared incumbent is the opponent. Because

the problem sits on the board rather than in the rules, one set of rules covers many kinds of problem.

4.1 Why games came first

Games became AI's laboratory because they supply four things at once: fixed rules, an impartial referee, a clear winner and cheap replay. Chess was chosen because it is "sharply defined both in allowed operations (the moves) and in the ultimate goal (checkmate)" [101], and checkers showed that a program could improve by playing itself [98]. Together these properties support repeated, reproducible evaluation. People scale judgment the same way, through prizes, leaderboards and the common task framework's "scoring referee" ([30]; Section 3). Invention has the first three properties once the problem and the solution are written down; cheap replay is a question of cost (Section 5).

4.2 The rules

Table 4 sets the game beside Go. Two rows differ in kind: an illegal move is played and lost, and the board holds the incumbent as well as the problem, because a move can only win against something.

Table 4: **Invention has every part of a board game, except cheap moves.** Go and the innovation game, element by element. Go's game length is from Silver et al. [105].

	Go	The innovation game
Board	Stones on a 19×19 grid	The encoded problem plus the incumbent
Move	Place one stone	Write one complete program, or one edit
Legal move	An empty point; no suicide	Parses, type-checks, runs within limits
Illegal move	Not allowed	Stillborn: an automatic loss
Referee	The rules of Go	The verifier, on hidden folds
Win	More territory	Valid, and beats the incumbent
Opponent	A past self	The incumbent, often a past self
Rating	Elo	Win rate and Elo across problems
Moves per game	About 150, each cheap	One, costly to judge

Formally, the game is a tuple of seven parts,

$$\mathscr{G} = (\mathscr{P}, \phi_{\mathrm{P}}, \mathscr{S}, \phi_{\mathrm{S}}, \mathscr{R}, r, \mathscr{O}).$$

Here ϕ_{P} and ϕ_{S} embed a problem $p \in \mathscr{P}$ and a program $w \in \mathscr{S}$ as z_p and z_w. The referee $\mathscr{R}$ reports a program's validity and score, the reward r orders moves, and the opponent $\mathscr{O}$ supplies the incumbent $\mathrm{inc}(p)$. The tuple extends Rice's model of algorithm selection [95]: the player writes the algorithm instead of picking it, and a declared incumbent sets the comparison standard.

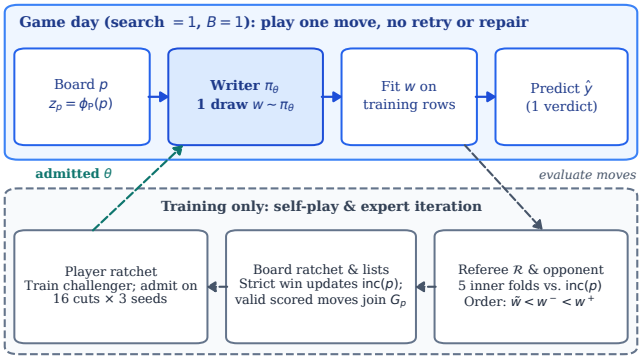


Figure 3: **Training plays the whole game; game day plays one move.** Schematic. Top row (blue): game day. Dashed box: training only. Outcomes are recorded; valid scored programs can enter the learning lists, while failed attempts still count against the budget.

A skeptic might ask whether a one-move game is a game at all. Formally it is a contextual bandit, with the board as context, the program as action and the verdict as reward [59]. It earns the name through an opponent that improves as the player does. The honest asymmetry is in the moves: Go offers about 250 legal moves per turn over about 150 turns, each free to check [105], while this game offers one move, costly to judge. Taste, which raises the chance q that one try wins (Equation (1)), therefore decides it.

Figure 3 draws the loop. Game day plays only its top row, so nothing can send a move back for another try; that is search $= 1$.

4.3 The board and the move

The board is the aim pillar of Section 2, made machine-readable: a summary of the training data, the objective and metric, and the constraints. Landmark scores from cheap learners [86] and learned tabular summaries are possible encodings. The reference checkpoint in Section 6.2 uses a frozen 2,048-value summary with a 30-second cap. A useful board should preserve distinctions relevant to algorithm choice, remain stable under irrelevant reorderings, exclude final evaluation information and have a measured cost. These are requirements to test, not properties guaranteed by naming an encoder. Whoever writes the board writes the aim. People still choose the problem, and the machine learns to answer it.

The move is where invention gets its language: one program in a typed language over learning primitives. The reference language uses a 401-token vocabulary, with up to six transforms chosen from 31 and one of 22 prediction heads (Section 6.3). Types remove whole classes of stillborn programs before they run [73]. A canonical form makes two spellings of one idea count once, as AutoML-Zero reused the score of any program whose outputs matched one already scored [94]. The language bounds what can be expressed; changing its primitives changes that space [9]. The reference grammar also bounds composition depth. Structural constraints can prevent

invalid programs; their effect on useful proposal quality must be measured (Section 5).

4.4 The referee and the opponent

The reward is an ordering first and a number second. Validity is a gate: any stillborn program scores below any valid one. Past the gate, programs rank by improvement over the incumbent on hidden folds. The reference promotion rule requires strict improvement under the same evaluation protocol and keeps the existing incumbent on a tie. Novelty never pays for losing: any novelty bonus (Section 5) must respect the order. For a stillborn program $\tilde{w}$, an also-ran w^- and a win w^+ on board p,

$$r(p,\tilde{w}) < r(p,w^-) < r(p,w^+).$$

The opponent is a ratcheting incumbent. In the reference procedure a program displaces a board's incumbent only by strict improvement on five inner folds. A retrained writer has a separate admission test on 16 held-out cuts with three seeds each (Section 6.5). These are distinct ratchets, as AlphaGo Zero's network admission was distinct from choosing a move during play [107]. The ratchet also makes this self-play. Subtracting the incumbent's score does not change which move is best; what matters is what promotion changes. It raises the bar, turns yesterday's winners into today's also-rans and so into hard negatives for the problem–program map (Section 6), and shifts which problems are worth playing.

4.5 One game, many boards

Binary and multiclass classification, regression, ranking, forecasting and relational prediction are prospective boards. Each needs a compatible encoding, program language, metric and validation protocol; a shared formal interface requires its own qualification for each task type. Beyond machine learning, examples of referees include sorting tests [68], evaluators for mathematical constructions [96] and accelerator-kernel timers [77]. These motivate extensions of the recipe, rather than results of this paper (Section 8).

Casting algorithm discovery as a game is not new, but earlier games wrote one family of problems into their rules, as AlphaTensor did for matrix multiplication and AlphaDev for sorting [35, 68]. AlphaZero used one algorithm for three games and trained a separate network for each [108]. Here the problem is an input on the board, so one player can learn many boards.

Key idea

A reliable referee makes a discipline a candidate for this game; transfer to that discipline still needs evidence.

4.6 One game played, and where the analogy breaks

The running example is the sealed problem chosen by the paper's stated rule, the median win (Section 7): **[TBD: dataset, task type, metric]**. The player encodes it in **[TBD: time]** and writes one program of **[TBD: size]**, which passes its checks. Fit in **[TBD: time]**, the program scores **[TBD: score]** on hidden folds against the incumbent's **[TBD: score]**: **[TBD: a win or an also-ran]**. Had it crashed, the game would simply have been lost, with no repair and no second try. The update is skipped because the problem is sealed: the verdict is reported, never learned from.

The analogy breaks in four places. **The referee is noisy**: picking a winner by a noisy validation score biases that score upward [17]. Strict numerical improvement alone does not quantify confidence; independent evaluation and an uncertainty protocol are needed to interpret it. **Moves are expensive**, each a fit and a cross-validation (Section 5). **Judges can be gamed**, as METR documented for frontier models in June 2025 [122]; Section 6 lists our safeguards. And **no algorithm wins everywhere**: averaged uniformly over all problems, no learner has lower error off the training set than another, ours included [126]. Real problems are structured, though, and a player that reads the board can choose its bias board by board. Section 7 reserves the tests of that hypothesis against fixed frontier-designed algorithms, with paired outcomes beside Elo summaries [33]. Non-transitive comparisons are one reason not to reduce the evidence to a single rating [4].

A game anyone can play is only useful if someone can afford to play it.

5 The Price of Play

What would it cost to play the innovation game? Under the planning assumptions below, generation and updates for a 30B-class open-weight writer cost about $12,000 in GPU time. This is a partial compute projection, not an observed price for a complete training run. Encoding, input tokens, repairs and referee execution must be added to the ledger. Cheap replay is the one property of Go that invention lacks (Section 4). Here we separate planning assumptions from the measurements still required in Section 7.

5.1 The bill in one line

For repeated attempts with a fixed win probability, expected cost per win is mean cost per attempt, repairs included, divided by that probability:

$$C_{\text{win}} = \frac{C_{\text{attempt}} + \mathbb{E}[n_{\text{repair}}]\, C_{\text{repair}}}{q}. \tag{2}$$

Here C_{attempt} is the cost of one try, including the referee's fit; C_{repair} is the cost of one repair turn, and $\mathbb{E}[n_{\text{repair}}]$ the mean number of repairs per attempt. The q is that of Equation (1), the probability that one try beats the incumbent: what sets the number of tries also sets the price. It factors as $q = q_{\text{run}}\, q_{\text{beat}}$, the share of moves that run times the share of those that win. A move that runs and loses still costs money, so we divide by q, not by the survival rate. Generating a token costs about 2 FLOPs per active parameter, and training on one about 6

Figure pending: figures/fig4_price.pdf

Figure 4: **Pilot cost measurements pending.** Planned panels: (a) the share of development proposals that run, score, beat the incumbent and survive re-verification, with the compute spent at each stage; (b) cost per verified win by writer size and reasoning effort. **[TBD: Supply pilot data, seeds and family-level uncertainty; identify settings with no verified win without assigning a finite cost per win.]**

[50]. We give self-hosted projections and dated API-price examples (Section 5.6).

The bill has two ledgers. Training pays for every attempted candidate, including failures; deployment pays for encoding, one proposal and its fitting procedure, with no repair. Fixed-weight agent loops instead buy evaluation during task-specific search; a trained proposer may amortize some of that work across later tasks. Whether the saving repays training requires a measured crossover, still pending in Section 7.

5.2 Wasted moves

An attempt can fail before or after it becomes a valid competitor. A stillborn attempt fails to produce a usable program under the declared protocol: generation is incomplete, parsing or type-checking fails, execution exceeds its limits or crashes, or outputs fail predeclared validity checks. An also-ran is valid but does not beat the incumbent. The planned funnel in Figure 4a traces those stages and the compute spent before each failed attempt stops. If 90% of attempts fail to improve the incumbent, the independent-try model requires ten attempts per improvement in expectation. A typed program language removes many stillborn moves by construction (Section 4), and those that remain usually fail before the costly fit. Whether also-rans dominate is a pilot question. Learning, a changed search procedure or a changed opponent can affect the measured win rate; isolate the effect attributed to the writer.

5.3 The correction tax and the reasoning dial

Small models are cheap per token and can be dear per win. Compute per token falls about 190-fold from a dense 1.5T writer to an 8B one, so the small model is cheaper per win unless its win rate per attempt is more than 190 times lower, repairs counted. That threshold flatters the small model: repair turns re-read a growing context, and a writer that needs three times the tokens per attempt breaks even at about 60, not 190.

Reasoning is the second dial. Reasoning tokens may cut stillborn moves, but each is billed as output. Figure 4b maps cost per win over writer size and reasoning effort. A U-shaped curve is the hypothesis; the measurement decides where, and whether, the minimum sits.

5.4 Rewarding useful hallucination

A pretrained writer pulls toward the textbook answer, which is history's taste. On a board whose incumbent is that answer, it is an also-ran by definition. A *useful hallucination* is a move the pretrained prior rates unlikely and the referee confirms (Box 3). Four counterweights push the writer toward such moves: a novelty bonus paid only inside the validity gate (Section 4), temperature and entropy schedules, a curriculum that starts past known solutions, and a quality-diversity archive [64, 74]. Section 6 states which of these EurekaZero uses. Novelty also raises waste, so exploration gets a stated share of games, **[TBD: X%]**.

Box 3: Why a referee makes hallucination safe

In text, a hallucination is an error nobody checks. In a game, an unfamiliar move is a conjecture, and the referee checks every one. Unfamiliar and verified is a discovery; unfamiliar and unchecked is confabulation. In March 2016, AlphaGo's own estimate put the chance that a human would play its Move 37 against Lee Sedol at about one in ten thousand [71], and AlphaGo won the game. A referee does not make strange moves good. It makes them cheap to try, and it is only as trustworthy as what it measures (Section 6).

5.5 How many games?

Three quantities differ by orders of magnitude. **Tries to find one improvement:** $1/q$ on average (Equation (1)). If tries were independent, $q = 1\%$ would need 69 tries for even odds and 299 for 95% confidence; tries from one writer are correlated, so these are guides, not guarantees. **Experience to train a transferable player:** AlphaGo Zero generated 4.9 million self-play games in about three days [107], and AlphaZero played 44 million games of chess in about nine hours [106, 108]. Our planning budget of 10,000 problems, 100 rounds and 8 attempts gives 8 million attempts, the same order. It counts work, not learning: problem families matter more, and pilot learning curves decide, read only within their measured range. **Precision to plan the test:** illustratively, 150 independent units at a win rate near 60% give an approximate 95% interval of ± 8 points. The actual study must compute uncertainty over independent families, rather than treating every dataset as an independent observation.

5.6 A run, priced

Table 5 prices an 8-million-attempt run for six writers: about $12,000 with a 30B-class writer and about $570,000 with a dense 1.5T one. The dense 1.5T rollouts take 1.2×10^{23} FLOPs, about 1% of a 10^{25}-FLOP pretraining run, a scale more than 30 public models had reached by mid-2025

[34, 93]. Mixture-of-experts (MoE) writers pay compute on their active parameters, such as the 32B of 1T in Kimi K2 [53] or the 37B of 671B in DeepSeek-V3 [27]. They need memory for all of them: 1T one-byte weights fill at least 13 H100s.

The referee runs on CPUs, at 10 to 100 CPU-seconds per move and \$0.015–\$0.045 per vCPU-hour [28]. For a full run that is \$330–\$990 at 10 seconds and \$3,300–\$9,900 at 100, enough to exceed an 8B writer's whole bill. List prices sit above our floor. On 30 October 2026, Together AI listed \$0.60 per million output tokens for gpt-oss-120B (5.1B active) and \$0.14 for an 8B Llama model [80, 117]: about 7 and 2 times our floor for the same model. The running example's single game cost **[TBD: \$c]**: **[TBD: t]** generated tokens and **[TBD: s]** CPU-seconds of referee time.

Key idea

Cost per verified improvement depends on both the cost of an attempt and the probability that it wins.

The table prices a fixed number of attempts, not a fixed number of wins. Converting those totals to cost per win requires a measured win rate. Under Equation (2)'s fixed-probability accounting, doubling q halves the expected cost per win while leaving the per-attempt bill unchanged.

The scaling scenarios in this section concern 8B–1.5T language-model writers producing 5,000 tokens per attempt; they do not price the reference checkpoint in Section 6. That checkpoint uses a compact network with 27.8 million parameters and at most 128 grammar-constrained tokens, with no repair in the proposed single-candidate deployment protocol. Its measured costs must be reported separately from the scaling projections.

The method's job is to improve proposal quality; whether it does so must be measured under the stated deployment budget.

6 EurekaZero: Learning Taste

How can evaluation feedback improve a proposal before it is tried? The design uses verified search as training experience for a program writer, with a separate problem–program map learning to rank scored programs. At deployment the writer submits one candidate. We describe the historical reference checkpoint below; the later adapter-training snapshot in Section 6.8 is separate evidence, not an evaluation of this checkpoint's one-proposal behavior. Learning to rank and learning to propose are distinct claims.

6.1 The player in one picture

In Figure 5 a frozen encoder reads the board and the program writer π_θ writes one complete program, as AlphaGo's policy network proposed a move [105]. The problem–program map v supplies a separate ranking score. In this reference checkpoint it does not condition or screen the writer's output, and is absent from deployment. Ranking a search's output does not by itself

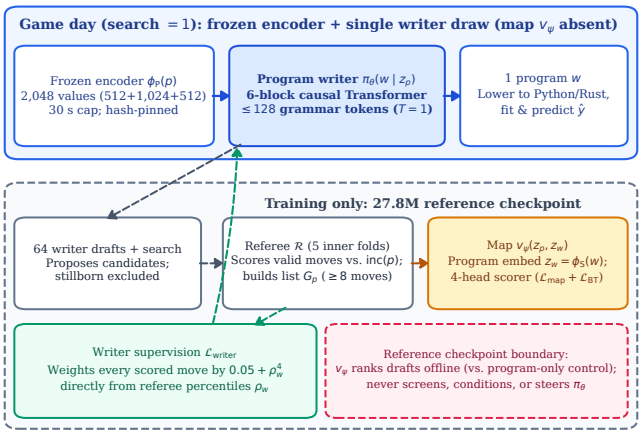


Figure 5: **A writer and a separate ranking map learn from scored programs.** Schematic. Top: game day, the whole of search = 1. Dashed box: training only. Search seeded with the writer's drafts proposes more programs, the referee scores them against the incumbent, and the scored lists retrain the writer (θ) and the map (ψ). Stillborn programs enter no list.

train or steer the proposer; search = 1 rests on the writer's single draw.

6.2 Reading the board

The reference checkpoint uses a frozen encoder to turn the board of Section 4 into 2,048 numbers, $z_p = \phi_P(p)$, in three blocks. The first summarizes a pretrained row encoder's embeddings across rows, independent of row order (512 values). The second projects the hidden state of a pretrained third-party tabular regressor that reads the training rows and labels (1,024; **[TBD: name, version, licence]**). The third passes the scaled targets through two frozen windows (512). The encoding contract permits training-side task evidence only; final held-out labels and their derived statistics must stay outside the input closure. A context's recorded declaration that no final target was used is part of the audit trail, not a substitute for checking that closure. It is fixed and pinned by hash in the reference procedure, rather than jointly trained with the writer. It is bounded at 30 seconds per problem, and a context with any non-finite value is refused rather than imputed (Appendix C). Whether problems with similar best programs sit together requires a separate representation study; Section 7.4 reserves that test.

6.3 Describing the move

A move is one program in the typed language of Section 4, spelled with a vocabulary of 401 tokens: up to six transforms from a set of 31, feeding one of 22 prediction heads. A grammar automaton masks every forbidden token at each step, the valid-only decoding of grammar-constrained generators [24, 58], so every finished program is grammatical. We credit that to the grammar, not to taste; taste must remove the moves that parse and then crash or lose. A program's identity is the

Table 5: **Projected generation and update costs for a fixed attempt budget.** These totals exclude other components of a full run; the number of verified wins remains to be measured. Projections before pilots; [low-cost, high-cost] in brackets. Base: 8 million attempts of 5,000 generated tokens; $2.92 per H100-hour on 29 October 2026 [81] [$2.50, $4.17; [116]]; 989.5 dense BF16 TFLOP/s per GPU [78]; generation at 20% of peak [30%, 10%], or 10% [20%, 7%] for memory-bound mixture-of-experts (MoE) writers; updates on 10% of tokens at 40% of peak [50%, 30%]. The 1.5T rows are scenarios. Excludes input tokens, repairs and the referee. $k: thousands of dollars; assumptions and numerical scenarios from the supplied cost drafts. **[TBD: Supply a reproducible calculation artifact before publication.]**

Program writer	Active params	GPU-h per 1,000 attempts	$ per 1,000 attempts	Rollouts, 8M attempts ($k)	Updates, full run ($k)
Small dense	8B	0.11	0.33	2.6 [1.5, 7.5]	0.4 [0.3, 0.7]
Mid dense	32B	0.45	1.31	10.5 [6.0, 30.0]	1.6 [1.1, 3.0]
Large dense	70B	0.98	2.87	23.0 [13.1, 65.6]	3.4 [2.4, 6.6]
1T MoE	32B	0.90	2.62	21.0 [9.0, 42.8]	1.6 [1.1, 3.0]
1.5T MoE (scenario)	50B	1.40	4.10	32.8 [14.0, 66.9]	2.5 [1.7, 4.7]
1.5T dense (scenario)	1.5T	21.05	61	492 [281, 1405]	73.8 [50.5, 140]

SHA-256 hash of its canonical form, so two spellings count once, and a hash of its predictions on fixed training probes flags different programs that compute the same function. Every move lowers to readable Python and Rust.

6.4 One map for problems and programs

The map reads a program as $z_w = \phi_S(w)$, the mean of learned token embeddings. It combines z_w with z_p three ways (as is, gated by the problem, and as a distance from a projection of it) and averages four heads into a score $v(z_p, z_w)$. It learns from lists: for each problem p with at least eight scored programs, G_p holds them and $\rho_w \in [0,1]$ is a program's percentile, 1 for the best. The map's softmax over the list is trained toward a target concentrated on the best [16]:

$$\mathscr{L}_{\text{map}}(p) = -\sum_{w \in G_p} \frac{e^{\rho_w/\tau}}{\sum_{u \in G_p} e^{\rho_u/\tau}} \log \frac{e^{v(z_p,z_w)}}{\sum_{u \in G_p} e^{v(z_p,z_u)}}, \qquad (3)$$

with $\tau = 0.25$. With a unique best program, the low-temperature limit concentrates the target on that program. This is a listwise ranking objective over programs scored on the same problem; the analogy to contrastive learning [92, 119] does not establish CLIP-style alignment across problems and programs. A Bradley–Terry term on the same lists sharpens the top [12]. Stillborn moves have no score, so this ranking objective covers only programs that run. A program-only head trained beside it tests whether the map reads the problem at all (Section 7).

Key idea

At search = 1 there is nothing left for a value network to judge. Taste has to live in the writer.

In this checkpoint the map never touches the writer's draw. It ranks the writer's drafts, a ranking we score against the referee, but drops none, and it does not condition, screen or steer the writer. The writer's learning targets come from referee scores directly. Thus ranking agreement with the referee, including comparison with the program-only control, can assess the map; attributing an improvement in proposals to the map would require an implemented influence path and a matched ablation (Section 7.4).

Algorithm 1 The self-play loop (training).

Require: training problems; serving writer π_θ and map v; incumbents $\text{inc}(p)$

for each round **do**
  for each sampled training problem p **do**
    $z_p \leftarrow \phi_P(p)$; draw 64 distinct legal programs from $\pi_\theta(\cdot \mid z_p)$ ▷ $T = 1$
    verified search, seeded with the drafts, proposes more programs ▷ the teacher
    $\mathscr{R}$ fits and scores each distinct program on 5 inner folds ▷ stillborn: no score
    if a program strictly beats $\text{inc}(p)$ **then** it becomes $\text{inc}(p)$ ▷ board ratchet
    add every scored program to the list G_p ▷ winners and also-rans
  train a challenger (θ', ψ') on the lists
  if it beats π_θ on held-out cuts **then** $(\theta, \psi) \leftarrow (\theta', \psi')$ ▷ player ratchet

6.5 The self-play loop

Training is expert iteration [1]: verified search is the teacher and the writer is the student (Algorithm 1). The writer imitates every scored program in proportion to the referee's verdict, $\mathscr{L}_{\text{writer}}(p) = -\sum_{w \in G_p} (0.05 + \rho_w^4) \log \pi_\theta(w \mid z_p)$, inside $\mathscr{L}_{\text{writer}} + 0.1\,\mathscr{L}_{\text{map}} + 0.25\,\mathscr{L}_{\text{BT}} + 0.1\,\mathscr{L}_{\text{runtime}} + 0.05\,\mathscr{L}_{\text{prog}}$ (Appendix C). Losing moves still teach: a median also-ran carries about a tenth of a winner's weight and is a negative for the map.

Two ratchets make this self-play rather than imitation of a fixed archive (Section 4). A program displaces a board's incumbent only by strictly beating it. A retrained writer is tested on 16 held-out cuts with three seeds each. It replaces the serving writer only if its mean log error ratio favors it, it

Algorithm 2 Game day in five lines (search = 1).

Require: new problem p with training rows and metric; frozen ϕ_{P} and π_θ

1: $z_p \leftarrow \phi_{\mathrm{P}}(p)$ ▷ read the board
2: $w \sim \pi_\theta(\cdot \mid z_p)$, one draw at $T = 1$ ▷ one move; no retry
3: $f \leftarrow \textsc{Fit}(w,$ training rows$)$ ▷ as any algorithm is fit
4: $\hat{y} \leftarrow f($new rows$)$ ▷ a stillborn w scores as a loss
5: **return** w, $\hat{y}$

wins at least as many cuts as it loses, and it fails no more often.

The historical reference. The writer is a decoder-only transformer (six blocks, width 512) that reads the context as a prefix token and writes at most 128 tokens; with the map, the network has 27.8 million parameters. The supplied source notes describe training on eight A100 GPUs in September 2026, with a corpus of about 10.2 million scored programs from 80 training datasets. **[TBD: Provide the checkpoint acceptance record and pin the reference runtime before final evaluation.]** These specifications identify the historical reference; they do not describe the later adapter snapshot or establish its deployment performance. Details belong in Appendix C.

6.6 Keeping the referee honest

A learner paid by a referee will learn the referee's flaws. In February 2025 Sakana AI walked back claims after its AI CUDA Engineer exploited the evaluation code [124]. In June 2025 METR reported frontier models patching their scorers [122], and 16% of 1,968 agent-benchmark tasks can be passed without being solved [131]. We require the evaluation protocol to withstand such attacks. Selection, promotion and final scoring must use separate data (inner folds, held-out cuts, a final fold scored once), with sealed meta-test families outside the learner's inputs. The proposed comparisons require sandboxed execution using seccomp and Landlock, with memory, swap and time limits. A planted label leak and a sealed canary must both be caught, and every reported winner must be re-run from its emitted code, with native and emitted predictions agreeing within $10^{-9}\max(1, |\hat{y}|)$. One earlier leaderboard entry for our search took, per dataset, the best of 63,108 test-scored evaluations; we exclude it as test selection (Appendix B).

6.7 Game day: search = 1

Algorithm 2 is the whole of game day: one encoder pass, one draw of at most 128 tokens at temperature 1 from a recorded seed, lowering to code, and the fit. The map is never run, and none of training's exploration programs is added. A draft that does not finish, crashes or times out is stillborn and scores as a loss; it is not replaced. Reported costs count the encoder and writer with the fit, and any later search is reported apart, as Silver et al. [107] reported AlphaGo Zero's raw network apart from its search (Section 7).

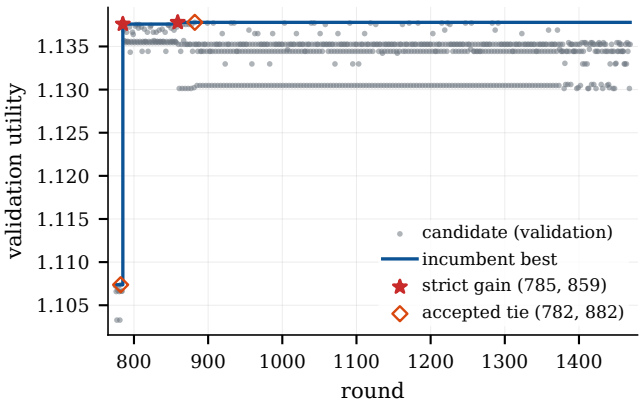


Figure 6: **Recorded validation trajectory.** Grey: candidate adapter utility. Blue: recorded best utility. Stars: strict increases in the recorded best; hollow diamonds: accepted ties at the existing best.

The deployment budget is a protocol constraint. Whether training improves the quality of that single proposal is a separate empirical question for Section 7.

6.8 Where training stands

What does the retained training record establish? These CSVs describe a later adapter-training snapshot, separate from the historical reference checkpoint above. Here an adapter is the trainable adjustment evaluated by the admission loop; the utilities alone do not identify its relationship to the historical writer and map. Utility denotes the logged scalar score, with larger values treated as better in the admission record. **[TBD: Document the adapter's architecture, score normalization and evaluation population before comparing this snapshot with other models.]**

The admission log contains 695 consecutive rounds. Four rows are marked accepted, but only two strictly increase the recorded best validation utility within this window. Its final value is 1.1378, with no later increase in the retained interval (Figure 6). Acceptance flags and improvements in that value are therefore distinct counts.

The more detailed summaries cover only 25 of those rounds. In that subset, training utility exceeds validation utility by 0.282–0.290 (Figure 7). This is a difference between reported aggregates on separate training and validation contexts; it does not isolate a cause or establish behavior in the missing summaries. The late validation plateau is evidence to explain, rather than proof of useful transfer. Encoder integrity and parameter updates require their own receipts; the plotted utility tables do not establish either.

Section 7 separates this development record from the independent comparisons needed to evaluate the invention claim.

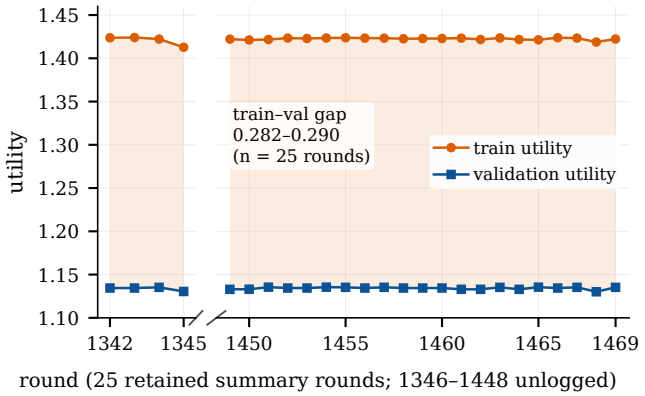


Figure 7: **Train and validation aggregates on retained summaries.** The 25 observations span rounds 1342–1469; a broken axis separates the observed blocks. Missing rounds are unobserved, and the reported utility gap covers only these summaries.

7 Results

Does training make the first proposed algorithm better on an unseen problem? The available training trajectory in Section 6.8 records validation behavior during learning. It does not answer the deployment question: those validation outcomes inform admission and are not an untouched test. This section specifies the comparisons still needed. **[TBD: Add independent evaluation evidence before making a performance claim.]**

7.1 Evaluation contract

Freeze the writer, permitted task inputs, output support, decoding rule and budgets before opening final outcomes. Separate unseen problem families from training and development, and report which task types were actually tested. Each EurekaZero submission follows Algorithm 2; failed generations and executions remain in the denominator. Report paired results and uncertainty at the family level, alongside encoding, generation, fitting and prediction costs. Disclose any internal model or hyperparameter selection in the emitted procedure, even when the writer submits only one outer candidate. **[TBD: Document the final problem inventory, family exclusions, evaluator, checkpoint, seeds, task metrics and failure accounting.]**

7.2 One proposal against fixed methods

The proposed comparison asks whether one task-conditioned program competes with methods that run unchanged across tasks. Frontier-designed algorithms must be fixed before final evaluation; record the originating models, prompts, dates and implementation checks. Pin the KumoRFM configuration [36], disclose its permitted inputs and fitting procedure, and compare under the same task metrics. Table 6 reserves the comparison; no winner is assumed.

Table 6: **Planned paired comparisons.** No outcomes are yet supplied. Report ties, failures and excluded cases alongside wins.

Comparator	Outcome and uncertainty
Claude-designed method	**[TBD: paired evaluation]**
Codex-designed method	**[TBD: paired evaluation]**
Gemini-designed method	**[TBD: paired evaluation]**
KumoRFM	**[TBD: paired evaluation]**

Table 7: **Map evidence still required.** Ranking quality and a change in first-proposal quality are separate outcomes.

Question	Required evidence
Does the map rank candidates?	**[TBD: held-out ranking study]**
Does it improve the writer?	**[TBD: matched causal ablation]**
Does it reduce execution failures?	**[TBD: failure-class comparison]**

7.3 The cost of more attempts

Compare the single proposal with agents allowed successive attempts and development feedback. Keep final outcomes unavailable to those agents and count complete candidates, repairs and all task-time computation. Plot performance against both attempts and cost; if an agent does not reach a reference performance within its budget, report that limit. **[TBD: Supply attempt curves, matched budgets, repeated seeds and uncertainty.]**

7.4 What the map contributes

A representation study should test whether nearby problem encodings share useful programs, with data disjoint from fitting the map. Ranking agreement should also be compared with the program-only head, so a global preference for common programs is distinguishable from reading the problem. **[TBD: Supply problem-similarity and program-only-control evaluations.]**

A ranking head can agree with a referee without improving the writer. In the checkpoint described in Section 6.4, the map neither conditions nor screens the writer's draw. Its ranking accuracy therefore cannot establish a reduction in wasted proposals. A causal ablation needs a declared route by which the map changes training, with the writer, training budget and deployment protocol otherwise matched. **[TBD: Identify an implemented influence path or limit the study to ranking.]**

7.5 Failure cases and a worked example

Describe unsuccessful proposals and coverage limits as well as wins. Assign each attempt one terminal status: incomplete generation, parse or type-checking refusal, execution or resource failure, invalid output under declared checks, and valid ties, losses and wins. Every attempted candidate contributes

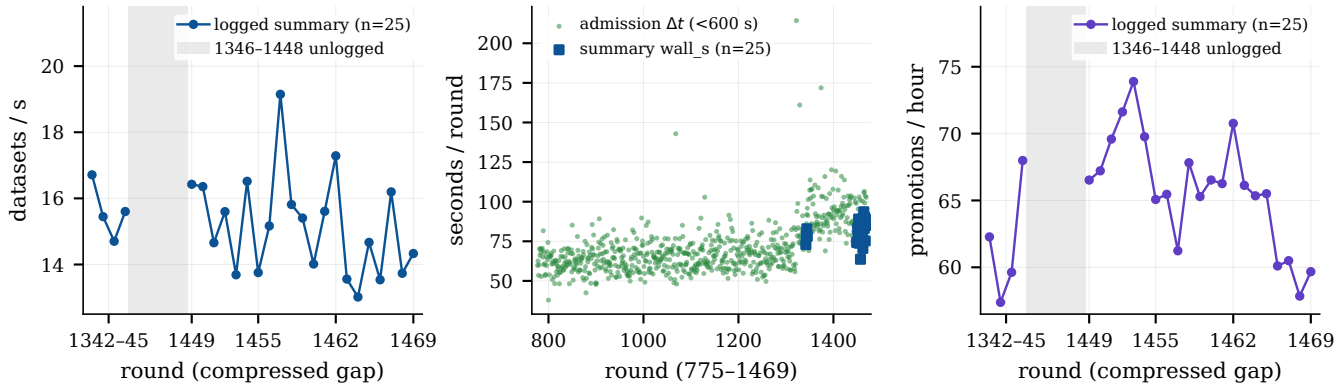


Figure 8: **Operating measurements from the retained snapshot.** Left: dataset throughput. Middle: filtered time between admission timestamps (green) and logged round wall time (blue squares); longer timestamp intervals are omitted as indicated in the legend. Right: promotion rate. These measurements describe the training loop, rather than the accuracy or cost of one-proposal deployment.

[TBD: Leaderboard: supply the evaluated methods, paired outcomes, rating protocol and uncertainty.]

Figure 9: **Leaderboard pending.** Elo is a summary of a specified comparison population, not a substitute for paired task results.

[TBD: Search curve: performance versus attempts and compute, including failed attempts and unreached targets.]

Figure 10: **Deployment search comparison pending.** The single-candidate condition fixes a budget; its measured quality remains to be established.

its incurred cost. Unresolved outcomes retain an explicit unknown status; fix their treatment and the validity rules before opening final outcomes. **[TBD: Supply attempt inventories and costs by outcome class, identifying unresolved cases.]**

Choose the worked example by a rule fixed before inspection, and show its emitted program, fitting procedure and comparison outcome; if the selected category is empty, report that fact. **[TBD: Supply the example selection rule, actual program and verified outcomes, with no replacement of failed attempts.]** These results will determine which interpretations in Section 8 the evidence can support.

8 Meaning

What would better first proposals on independent new families establish, and where would that claim stop? The answer depends on the independent results in Section 7. The recorded training trajectory alone cannot establish transfer, useful new composition or autonomous scientific discovery. **[TBD: Reconcile every conclusion below with the completed evaluation.]**

8.1 Learning to select and learning to invent

A better task-dependent choice from a fixed program archive demonstrates selection. A useful composition absent from the training archive is a stronger result, and requires an output space that permits it. Token-by-token decoding does not distinguish the two: report the admissible output space, the training archive and the relation of each proposed program to that archive. Absence from an archive establishes novelty relative to that archive, rather than to the scientific literature. Different program spellings can also express the same computation. Claims of invention therefore need comparisons of program behavior and known methods, alongside predictive quality. **[TBD: State which claim the evaluated checkpoint and its outputs actually support.]**

8.2 Moving judgment across problems

If learning improves first proposals on new families, it would support the hypothesis that training evaluations can be retained as a transferable proposal capability. It would not show that one proposal wins on every problem. Test-time search may still improve a learned writer, and its value should be compared with the training cost it takes to improve that writer. **[TBD: Use the measured search curves and full cost account to bound this claim.]**

8.3 The referee defines the boundary

The framework optimizes what its referee measures. Repeated development comparisons can overfit that referee, while a restricted language can hide useful algorithms from the learner. An admission based on a higher development score records a selection decision. It does not quantify uncertainty or guarantee improvement in the target population. The independent evaluation must keep the chosen checkpoint fixed and report paired uncertainty, including failures and ties. Task representations, training families and runtime limits further bound the result. A referee for a new scientific domain would need its

own validity and cost evidence; analogy to a game supplies neither. **[TBD: Report observed failures, distribution shifts and remaining tests of the referee.]**

8.4 A bounded claim about self-improvement

Improving an external algorithm, improving the proposer and improving the rate at which the proposer learns are distinct claims. Evidence for the first does not establish the others. An optimizer trace with verified parameter changes can establish that training occurred; a lower training objective measures the fitting criterion. Neither alone shows that first proposals improved on unseen families. People still choose the task, metric, representation and resource limits. **[TBD: State the strongest supported level of improvement and leave untested extensions as future work.]**

The contribution to assess is concrete: whether retained evaluation experience improves the first executable proposal on a new problem, under a declared budget and an independently scored test.

A Historical notes

Section 3 traces the four pillars of Section 2 through the milestones directly needed for our argument: algorithm selection [95], typed genetic programming and evolutionary search [55, 73, 94], self-play and expert iteration [1, 98, 107], and task-conditioned amortization across tabular problems [36, 46, 75]. **[TBD: Expand with the full chronological survey table and domain-by-domain referee inventory.]**

B Incident log and audit safeguards

Section 6.6 requires that evaluation artifacts violating held-out isolation be recorded and excluded rather than silently dropped:

- **Excluded test-selected search baseline.** An earlier internal leaderboard entry took, per dataset, the best of 63,108 test-scored evaluations across 42 datasets. Because selecting among candidates using test-fold scores biases the reported metric upward [17], that entry is excluded from all comparative claims.
- **Input-closure and canary checks.** The evaluation harness enforces three automated regression checks: an inner-fold label-poison test, a sealed canary check, and a train-only input-closure audit verifying that held-out test targets and their derived statistics do not enter board encoding or candidate generation. **[TBD: Attach final pre-registration and audit receipts before benchmark release.]**

C Training details

This appendix records the implementation specifications referenced in Section 6 for the historical reference checkpoint (separate from the later adapter snapshot in Section 6.8):

- **Board encoder blocks (Section 6.2).** The frozen 2,048-value context $z_p = \phi_{\mathrm{P}}(p)$ concatenates three blocks: (i) 512 values pooling a 128-dimensional pretrained row encoder across training rows by mean, standard deviation, and 10th and 90th percentiles; (ii) 1,024 values from a projected third-party tabular regressor hidden state capped at 768 context rows and 256 query rows (**[TBD: name, version, and licence]**); and (iii) 512 values from two frozen target-summary windows initialized with fixed seed 17. Any context with a non-finite entry is refused rather than imputed. Across eight reference encoding jobs, measured wall-clock latency ranged from 11.7 s to 20.8 s within the 30 s cap.
- **Objective terms and parameter split (Sections 6.4 and 6.5).** Lists G_p require at least eight valid scored programs per problem, with percentile $\rho_w = 1 - \mathrm{rank}/(|G_p| - 1)$ and writer sequence weight $0.05 + \rho_w^4$. In the combined objective $\mathscr{L}_{\mathrm{writer}} + 0.1\,\mathscr{L}_{\mathrm{map}} + 0.25\,\mathscr{L}_{\mathrm{BT}} + 0.1\,\mathscr{L}_{\mathrm{runtime}} + 0.05\,\mathscr{L}_{\mathrm{prog}}$, the pairwise Bradley–Terry term weights pairs by their score gap, doubled when either program is at or above the 0.90 percentile. The network has 27,796,494 parameters in total (six decoder blocks, width 512, four attention heads, prefix context token, and maximum length 128 tokens over the 401-token grammar); approximately 1.05 million parameters belong to training-time ranking and control heads that are absent at deployment.
- **Optimization notes (Section 6.5).** The supplied source notes describe an AdamW run (learning rate 10^{-4}, weight decay 0.01, gradient-norm clip 1.0, global batch size 2,048) on eight A100 GPUs in September 2026 over 10,214,332 scored programs from 80 training datasets (7,877 lists; 8,894 contexts). These historical reference specifications are separate from the later adapter snapshot in Section 6.8. **[TBD: Provide the checkpoint acceptance record and held-out promotion receipt before final evaluation.]**
- **Execution sandbox and parity (Section 6.6).** Refereed evaluation isolates candidate execution with seccomp and Landlock, enforces cgroup memory limits with swap disabled, and requires native and emitted-code predictions to agree within $10^{-9}\max(1, |\hat{y}|)$.

References

[1] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, volume 30, pages 5360–5370, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/d8e1344e27a5b08cdfd5d027d9b8d6de-Paper.pdf.

[2] Pierre Azoulay, Christian Fons-Rosen, and Joshua S. Graff Zivin. Does science advance one funeral at a time? *American Economic Review*, 109(8):2889–2920, 2019. doi: 10.1257/aer.20161574.

[3] Luke Bailey, Kaiyue Wen, Kefan Dong, Tatsunori Hashimoto, and Tengyu Ma. Scaling self-play with self-guidance. arXiv:2604.20209, April 2026.

[4] David Balduzzi, Karl Tuyls, Julien Pérolat, and Thore Graepel. Re-evaluating evaluation. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, pages 3268–3279. Curran Associates, 2018. URL https://arxiv.org/abs/1806.02643.

[5] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. DeepCoder: Learning to write programs. In *International Conference on Learning Representations (ICLR)*, 2017.

[6] Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957.

[7] James Bennett and Stan Lanning. The Netflix prize. In *Proceedings of KDD Cup and Workshop 2007*, 2007.

[8] Nicholas Bloom, Charles I. Jones, John Van Reenen, and Michael Webb. Are ideas getting harder to find? *American Economic Review*, 110(4):1104–1144, 2020. doi: 10.1257/aer.20180338.

[9] Margaret A. Boden. *The Creative Mind: Myths and Mechanisms*. Weidenfeld & Nicolson, London, 1990.

[10] Margaret A. Boden. Creativity and artificial intelligence. *Artificial Intelligence*, 103(1–2):347–356, 1998. doi: 10.1016/S0004-3702(98)00055-1.

[11] Jorge Luis Borges. La biblioteca de Babel. In *El jardín de senderos que se bifurcan*. Sur, Buenos Aires, 1941. Printed 30 December 1941, issued 1942 (LCCN 43002084). English translation by James E. Irby, "The Library of Babel", in *Labyrinths*, New Directions, 1962.

[12] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[13] Russell Brandom. What happens when AI starts building itself? TechCrunch, 14 May 2026. URL https://techcrunch.com/2026/05/14/what-happens-when-ai-starts-building-itself/.

[14] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. OpenAI technical report, February 2024. URL https://openai.com/index/video-generation-models-as-world-simulators/.

[15] Donald T. Campbell. Blind variation and selective retention in creative thought as in other knowledge processes. *Psychological Review*, 67(6):380–400, 1960. doi: 10.1037/h0040373.

[16] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. Learning to rank: From pairwise approach to listwise approach. In *Proceedings of the 24th International Conference on Machine Learning (ICML)*, pages 129–136, 2007. doi: 10.1145/1273496.1273513.

[17] Gavin C. Cawley and Nicola L. C. Talbot. On overfitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[18] William G. Chase and Herbert A. Simon. Perception in chess. *Cognitive Psychology*, 4(1):55–81, 1973. doi: 10.1016/0010-0285(73)90004-2.

[19] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.

[20] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[21] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, 2024.

[22] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.

[23] Seth Cooper, Firas Khatib, Adrien Treuille, Janos Barbero, Jeehyung Lee, Michael Beenen, Andrew Leaver-Fay, David Baker, Zoran Popović, and Foldit Players. Predicting protein structures with a multiplayer online game. *Nature*, 466(7307):756–760, 2010. doi: 10.1038/nature09304.

[24] Hanjun Dai, Yingtao Tian, Bo Dai, Steven Skiena, and Le Song. Syntax-directed variational autoencoder for structured data. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1802.08786.

[25] Charles Darwin. *On the Origin of Species by Means of Natural Selection, or the Preservation of Favoured Races in the Struggle for Life*. John Murray, London, 1859. URL http://darwin-online.org.uk/content/frameset?itemID=F373&viewtype=text&pageseq=1.

[26] Adriaan D. de Groot. *Thought and Choice in Chess*. Mouton, The Hague, 1965.

[27] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, et al. DeepSeek-V3 technical report. arXiv:2412.19437, 2024. URL https://arxiv.org/abs/2412.19437.

[28] DoiT. AWS EC2 c7i.large: Spot, on-demand and savings plan pricing in us-east-1. DoiT Cloud Compute Pricing, 2026. URL https://www.doit.com/compute/spot/us-east-1/c7i.large.

[29] Kefan Dong and Tengyu Ma. STP: Self-play LLM theorem provers with iterative conjecturing and proving. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, 2025. URL https://proceedings.mlr.press/v267/dong25h.html.

[30] David Donoho. 50 years of data science. *Journal of Computational and Graphical Statistics*, 26(4):745–766, 2017. doi: 10.1080/10618600.2017.1384734.

[31] Shangheng Du, Xiangchao Yan, Jinxin Shi, Zongsheng Cao, Shiyang Feng, Zichen Liang, Boyuan Sun, Tianshuo Peng, Yifan Zhou, Xin Li, Jie Zhou, Liang He, Bo Zhang, and Lei Bai. MLEvolve: A self-evolving framework for automated machine learning algorithm discovery. arXiv:2606.06473, June 2026.

[32] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 835–850, 2021. doi: 10.1145/3453483.3454080.

[33] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco, New York, 1978.

[34] Epoch AI. Data on notable AI models. Epoch AI Dataset, 2026. URL https://epoch.ai/data/ai-models.

[35] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, and Pushmeet Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022. doi: 10.1038/s41586-022-05172-4.

[36] Matthias Fey, Vid Kocijan, Federico Lopez, Jan Eric Lenssen, and Jure Leskovec. KumoRFM: A relational foundation model. Kumo Technical Report, May 2025. URL https://docs.nvidia.com/sdgm/research/concepts-models/kumorfm-paper.

[37] R. M. Friedberg. A learning machine: Part I. *IBM Journal of Research and Development*, 2(1):2–13, 1958. doi: 10.1147/rd.21.0002.

[38] Irving John Good. Speculations concerning the first ultraintelligent machine. In Franz L. Alt and Morris Rubinoff, editors, *Advances in Computers*, volume 6, pages 31–88. Academic Press, 1966. doi: 10.1016/S0065-2458(08)60418-0.

[39] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, et al. Accelerating scientific discovery with Co-Scientist. *Nature*, 655(8122):487–496, 2026. doi: 10.1038/s41586-026-10644-y. Preprint: arXiv:2502.18864 (2025), "Towards an AI co-scientist".

[40] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 317–330, 2011. doi: 10.1145/1926385.1926423.

[41] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.

[42] Richard W. Hamming. You and your research. Bell Communications Research Colloquium Seminar, Morris Research and Engineering Center, Morristown, NJ, 1986. URL https://hdl.handle.net/10945/37504. Talk given 7 March 1986; transcribed by J. F. Kaiser.

[43] Demis Hassabis and Alex Kantrowitz. Google DeepMind CEO Demis Hassabis: The path to AGI, LLM creativity, and Google smart glasses. Big Technology Podcast, 23 January 2025. URL https://www.bigtechnology.com/p/google-deepmind-ceo-demis-hassabis.

[44] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre.

Training compute-optimal large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, volume 35, pages 30016–30030, 2022. URL https://arxiv.org/abs/2203.15556.

[45] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975.

[46] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. doi: 10.1038/s41586-024-08328-6.

[47] Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026. doi: 10.1038/s41586-025-09833-y. Published online 12 November 2025.

[48] Benjamin F. Jones. The burden of knowledge and the "death of the Renaissance man": Is innovation getting harder? *Review of Economic Studies*, 76(1):283–317, 2009. doi: 10.1111/j.1467-937X.2008.00531.x.

[49] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

[50] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. arXiv:2001.08361, 2020. URL https://arxiv.org/abs/2001.08361.

[51] Andrej Karpathy. `autoresearch`: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 6 March 2026. URL https://github.com/karpathy/autoresearch.

[52] Firas Khatib, Frank DiMaio, Foldit Contenders Group, Foldit Void Crushers Group, Seth Cooper, et al. Crystal structure of a monomeric retroviral protease solved by protein folding game players. *Nature Structural & Molecular Biology*, 18(10):1175–1177, 2011. doi: 10.1038/nsmb.2119.

[53] Kimi Team. Kimi K2: Open agentic intelligence. arXiv:2507.20534, 2025. URL https://arxiv.org/abs/2507.20534.

[54] Arthur Koestler. *The Act of Creation*. Hutchinson, London, 1964.

[55] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, 1992.

[56] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems 25 (NeurIPS 2012)*, pages 1097–1105, 2012. URL https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.

[57] Thomas S. Kuhn. *The Structure of Scientific Revolutions*. University of Chicago Press, Chicago, 1962.

[58] Matt J. Kusner, Brooks Paige, and José Miguel Hernández-Lobato. Grammar variational autoencoder. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1945–1954, 2017. URL https://proceedings.mlr.press/v70/kusner17a.html.

[59] Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, Cambridge, 2020. doi: 10.1017/9781108571401.

[60] Yann LeCun and Alex Kantrowitz. Why can't AI make its own discoveries? — with Yann LeCun. Big Technology Podcast, 19 March 2025. URL https://podscripts.co/podcasts/big-technology-podcast/why-cant-ai-make-its-own-discoveries-with-yann-lecun.

[61] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.

[62] Jeehyung Lee, Wipapat Kladwang, Minjae Lee, Daniel Cantu, Martin Azizyan, Hanjoo Kim, Alex Limpaecher, Sungroh Yoon, Adrien Treuille, Rhiju Das, and EteRNA Participants. RNA design rules from a massive open laboratory. *Proceedings of the National Academy of Sciences*, 111(6):2122–2127, 2014. doi: 10.1073/pnas.1313039111.

[63] Young-Jun Lee, Seungone Kim, Minki Kang, Alistair Cheong Liang Chuen, Zerui Chen, Seungho Han, Taehee Jung, and Dongyeop Kang. Evolution fine-tuning: Learning to discover across 371 optimization tasks. arXiv:2606.29082, June 2026.

[64] Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011. doi: 10.1162/EVCO_a_00025.

[65] Douglas B. Lenat. *AM: An Artificial Intelligence Approach to Discovery in Mathematics as Heuristic Search*. PhD thesis, Stanford University, 1976. Stanford AI Lab Memo AIM-286; report STAN-CS-76-570.

[66] Douglas B. Lenat. EURISKO: A program that learns new heuristics and domain concepts. The nature of heuristics III: Program design and results. *Artificial*

Intelligence, 21(1–2):61–98, 1983. doi: 10.1016/S0004-3702(83)80005-8.

[67] Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. Towards end-to-end automation of AI research. *Nature*, 651(8107):914–919, 2026. doi: 10.1038/s41586-026-10265-5. Preprint: arXiv:2408.06292 (2024), "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery".

[68] Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, Thomas Köppe, Kevin Millikin, Stephen Gaffney, Sophie Elster, Jackson Broshear, Chris Gamble, Kieran Milan, Robert Tung, Minjae Hwang, Taylan Cemgil, Mohammadamin Barekatain, Yujia Li, Amol Mandhane, Thomas Hubert, Julian Schrittwieser, Demis Hassabis, Pushmeet Kohli, Martin Riedmiller, Oriol Vinyals, and David Silver. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618(7964):257–263, 2023. doi: 10.1038/s41586-023-06004-9.

[69] Henry Massalin. Superoptimizer: A look at the smallest program. In *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)*, pages 122–126, 1987. doi: 10.1145/36206.36194.

[70] Amil Merchant, Simon Batzner, Samuel S. Schoenholz, Muratahan Aykol, Gowoon Cheon, and Ekin Dogus Cubuk. Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85, 2023. doi: 10.1038/s41586-023-06735-9.

[71] Cade Metz. In two moves, AlphaGo and Lee Sedol redefined the future. *Wired*, 16 March 2016. URL https://www.wired.com/2016/03/two-moves-alphago-lee-sedol-redefined-future/.

[72] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

[73] David J. Montana. Strongly typed genetic programming. *Evolutionary Computation*, 3(2):199–230, 1995. doi: 10.1162/evco.1995.3.2.199.

[74] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015. URL https://arxiv.org/abs/1504.04909.

[75] Andreas Müller, Carlo Curino, and Raghu Ramakrishnan. MotherNet: Fast training and inference via hyper-network transformers. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2312.08598.

[76] Jens Nielsen and Jay D. Keasling. Engineering cellular metabolism. *Cell*, 164(6):1185–1197, 2016. doi: 10.1016/j.cell.2016.02.004.

[77] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131, 2025. URL https://arxiv.org/abs/2506.13131.

[78] NVIDIA. NVIDIA H100 tensor core GPU: Specifications. NVIDIA Data Center Documentation, 2026. URL https://www.nvidia.com/en-us/data-center/h100/.

[79] Junhyuk Oh, Gregory Farquhar, Iurii Kemaev, Dan A. Calian, Matteo Hessel, Luisa Zintgraf, Satinder Singh, Hado van Hasselt, and David Silver. Discovering state-of-the-art reinforcement learning algorithms. *Nature*, 648(8093):312–319, 2025. doi: 10.1038/s41586-025-09761-x.

[80] OpenAI. gpt-oss-120b. OpenAI developer documentation, 2025. URL https://developers.openai.com/api/docs/models/gpt-oss-120b.

[81] Ornn. Ornn compute price index (OCPI): Live GPU rental prices. Ornn Market Data, 2026. URL https://data.ornn.com/.

[82] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, volume 35, pages 27730–27744, 2022.

[83] Michael Park, Erin Leahey, and Russell J. Funk. Papers and patents are becoming less disruptive over time. *Nature*, 613(7942):138–144, 2023. doi: 10.1038/s41586-022-05543-x.

[84] Dwarkesh Patel and Dario Amodei. Dario Amodei (Anthropic CEO) — the hidden pattern behind every AI breakthrough. Dwarkesh Podcast, 8 August 2023. URL https://www.dwarkesh.com/p/dario-amodei.

[85] Charles Sanders Peirce. *Collected Papers of Charles Sanders Peirce, Volume V: Pragmatism and Pragmaticism*. Harvard University Press, Cambridge, MA, 1934. Edited by Charles Hartshorne and Paul Weiss.

[86] Bernhard Pfahringer, Hilan Bensusan, and Christophe Giraud-Carrier. Meta-learning by landmarking various learning algorithms. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML)*, pages 743–750. Morgan Kaufmann, 2000.

[87] Henri Poincaré. Mathematical creation. *The Monist*, 20(3):321–335, 1910. doi: 10.1093/monist/20.3.321. Translated by George Bruce Halsted.

[88] Michael Polanyi. *The Tacit Dimension*. Doubleday, Garden City, NY, 1966.

[89] George Pólya. *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, Princeton, NJ, 1945.

[90] Karl R. Popper. *The Logic of Scientific Discovery*. Hutchinson, London, 1959.

[91] Karl R. Popper. *Conjectures and Refutations: The Growth of Scientific Knowledge*. Routledge & Kegan Paul, London, 1963.

[92] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, 2021. URL https://proceedings.mlr.press/v139/radford21a.html.

[93] Robi Rahman, Lovis Heindrich, David Owen, and Luke Emberson. Over 30 AI models have been trained at the scale of GPT-4. Epoch AI Data Insight, 6 June 2025. URL https://epoch.ai/data-insights/models-over-1e25-flop.

[94] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 8007–8019. PMLR, 2020. URL https://proceedings.mlr.press/v119/real20a.html.

[95] John R. Rice. The algorithm selection problem. In Morris Rubinoff and Marshall C. Yovits, editors, *Advances in Computers*, volume 15, pages 65–118. Academic Press, New York, 1976. doi: 10.1016/S0065-2458(08)60520-3.

[96] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[97] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, et al. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015. doi: 10.1007/s11263-015-0816-y.

[98] Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959. doi: 10.1147/rd.33.0210.

[99] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. arXiv:cs/0309048, 2003.

[100] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. doi: 10.1038/s41586-020-03051-4.

[101] Claude E. Shannon. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950. doi: 10.1080/14786445008521796.

[102] Minkyu Shin, Jin Kim, Bas van Opheusden, and Thomas L. Griffiths. Superhuman artificial intelligence can improve human decision-making by increasing novelty. *Proceedings of the National Academy of Sciences*, 120(12):e2214840120, 2023. doi: 10.1073/pnas.2214840120.

[103] Chenglei Si, Diyi Yang, and Tatsunori Hashimoto. Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=M23dTGWCZy.

[104] Chenglei Si, Tatsunori Hashimoto, and Diyi Yang. The ideation–execution gap: Execution outcomes of LLM-generated versus human research ideas. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=Fllp8l6Puy.

[105] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.

[106] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc

Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv:1712.01815, 2017. URL https://arxiv.org/abs/1712.01815.

[107] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017. doi: 10.1038/nature24270.

[108] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404.

[109] Dean Keith Simonton. Creativity and discovery as blind variation: Campbell's (1960) BVSR model after the half-century mark. *Review of General Psychology*, 15(2):158–174, 2011. doi: 10.1037/a0022912.

[110] Richard Socher. AI's biggest breakthrough won't be a chatbot. Next Big Idea Club, Book Bites, 8 October 2026. URL https://nextbigideaclub.com/magazine/ais-biggest-breakthrough-wont-chatbot-bookbite/61534/.

[111] Richard Socher. *The Eureka Machine: Why AI Is the Key to Unlocking a New Era of Scientific Discoveries*. PublicAffairs, New York, 2026. ISBN 9781541705708. URL https://www.hachettebookgroup.com/titles/richard-socher/the-eureka-machine/9781541705708/.

[112] Jonathan M. Stokes, Kevin Yang, Kyle Swanson, Wengong Jin, Andres Cubillos-Ruiz, Nina M. Donghia, et al. A deep learning approach to antibiotic discovery. *Cell*, 180(4):688–702.e13, 2020. doi: 10.1016/j.cell.2020.01.021.

[113] Anja Surina, Amin Mansouri, Lars Quaedvlieg, Amal Seddas, Maryna Viazovska, Emmanuel Abbe, and Caglar Gulcehre. Algorithm discovery with LLMs: Evolutionary search meets reinforcement learning. arXiv:2504.05108, 2025.

[114] Richard S. Sutton. The bitter lesson. Essay, 13 March 2019. URL http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

[115] Gerald Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995. doi: 10.1145/203330.203343.

[116] Thunder Compute. AI GPU rental market trends (October 2026): Complete industry analysis. Thunder Compute Industry Report, 1 October 2026. URL https://www.thundercompute.com/blog/ai-gpu-rental-market-trends.

[117] Together AI. Serverless model pricing. Together AI Documentation, 2026. URL https://www.together.ai/pricing.

[118] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.

[119] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding, 2018. URL https://arxiv.org/abs/1807.03748.

[120] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 5998–6008, 2017.

[121] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019. doi: 10.1038/s41586-019-1724-z.

[122] Sydney Von Arx, Lawrence Chan, and Beth Barnes. Recent frontier models are reward hacking. METR blog, 5 June 2025. URL https://metr.org/blog/2025-06-05-recent-reward-hacking/.

[123] Graham Wallas. *The Art of Thought*. Jonathan Cape, London, 1926.

[124] Kyle Wiggers. Sakana walks back claims that its AI can dramatically speed up model training. TechCrunch, 21 February 2025. URL https://techcrunch.com/2025/02/21/sakana-walks-back-claims-that-its-ai-can-dramatically-speed-up-model-training/.

[125] Thomas Wolf. The Einstein AI model. Technical blog essay, March 2025. URL https://thomwolf.io/blog/scientific-ai.html.

[126] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.

[127] Mert Yuksekgonul, Daniel Koceja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz,

Yejin Choi, James Zou, Carlos Guestrin, and Yu Sun. Learning to discover at test time. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.

[128] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling (COLM)*, 2024.

[129] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. arXiv:2505.22954, 2025.

[130] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. arXiv:2505.03335, 2025.

[131] Ziqian Zhong, Ivgeni Segal, Ivan Bercovich, Shashwat Saxena, Kexun Zhang, and Aditi Raghunathan. Hardening agent benchmarks with adversarial hacker-fixer loops, 2026. URL https://arxiv.org/abs/2606.08960.

[132] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017.